



طراحی سامانه‌های میکروسرویس

نیم‌سال دوم ۱۴۰۴-۰۵

استاد: دکتر علی ابریشمی - سیدحسین میرحسینی

دانشکده‌ی مهندسی کامپیوتر

پاسخ‌دهنده: معین آعلی - ۴۰۱۱۰۵۵۶۱

تمرین اول

فهرست مسائل

۱		مسئله ۱
۱		الف
۲		ب
۲		ج
۳		مسئله ۲
۶		مسئله ۳
۷		مسئله ۴
۸		مسئله ۵
۸		شباهت‌های بین دو الگوی Clean Architecture و MVVM
۸		نحوه ادغام و ترکیب دو الگو توسط نویسنده
۸		پروژه‌های مناسب برای استفاده از این الگوی ترکیبی و دلایل آن

پاسخ مسئله‌ی ۱.

الف

موجودیت یا Entity نماینده‌ی یک داده‌ی واقعی در سیستم و پایگاه داده است. این کلاس‌ها معمولاً مستقیماً با جدول‌های دیتابیس در ارتباط هستند و دارای ID می‌باشند. همچنین ممکن است شامل منطق و رفتار مرتبط با دامنه‌ی مسئله باشند.

مثال:

```
class User {  
    Long id;  
    String name;  
    String password;  
}
```

DTO یا Data Transfer Object شیئی است که فقط برای انتقال داده بین لایه‌های مختلف سیستم استفاده می‌شود. DTO معمولاً منطق خاصی ندارد و فقط شامل اطلاعات موردنیاز است.

مثال:

```
class CreateUserDto {  
    String name;  
    String password;  
}
```

تفاوت اصلی این است که Entity نماینده‌ی داده‌ی اصلی سیستم و مرتبط با دیتابیس است؛ اما DTO فقط برای انتقال داده و کاهش وابستگی یا افزایش امنیت استفاده می‌شود.

ب

اصل CCP بیان می‌کند که کلاس‌هایی که به دلیل مشابهی تغییر می‌کنند، بهتر است در یک پکیج یا ماژول قرار بگیرند. به بیان ساده، اگر چند کلاس معمولاً هم‌زمان تغییر می‌کنند، باید کنار یکدیگر باشند تا مدیریت تغییرات ساده‌تر شود. برای مثال، کلاس‌های مربوط به فاکتور مانند: Invoice, InvoiceService, InvoiceValidator بهتر است در یک پکیج مشترک قرار بگیرند؛ زیرا تغییر قوانین فاکتور احتمالاً باعث تغییر همه‌ی آن‌ها می‌شود. مزیت این اصل:

- کاهش اثر تغییرات روی بخش‌های دیگر
- افزایش نگهداری‌پذیری پروژه
- ساده‌تر شدن توسعه و تست

ج

Decoupling به معنی کاهش وابستگی بین بخش‌های مختلف سیستم است. هرچه اجزای سیستم مستقل‌تر باشند، توسعه و نگهداری ساده‌تر خواهد بود. منظور از سطح Deployment این است که اجزای سیستم چگونه منتشر و اجرا می‌شوند. در یک سیستم کاملاً Decoupled، هر بخش می‌تواند به صورت مستقل Deploy شود. برای مثال در معماری Microservice سرویس کاربران و سرویس پرداخت و سرویس سفارش هرکدام جداگانه اجرا و منتشر می‌شوند. اما در معماری Monolith، معمولاً کل سیستم باید با هم Deploy شود. مزایای Deployment مستقل:

- سرعت بیشتر در انتشار تغییرات
- کاهش وابستگی بین نسخه‌ها
- امکان توسعه‌ی مستقل توسط تیم‌ها
- کاهش ریسک هنگام انتشار تغییرات

پاسخ مسئله‌ی ۲.

الهام‌گیری Architecture Clean از اصل DIP

اصل وارونگی وابستگی شامل دو بند اساسی است:

۱. ماژول‌های سطح بالا نباید به ماژول‌های سطح پایین وابسته باشند؛ هر دو باید به انتزاع وابسته باشند.
 ۲. انتزاع‌ها نباید وابسته به جزئیات باشند، بلکه جزئیات (پیاپیاده‌سازی‌های واقعی) باید وابسته به انتزاع‌ها باشند.
- در Clean Architecture، نرم‌افزار به لایه‌های متمرکزی تقسیم می‌شود که هسته مرکزی آن شامل بالاترین سطح از سیاست‌ها و منطق تجاری برنامه (Domain و Use Cases) است، و لایه‌های بیرونی شامل ابزارها و مکانیزم‌های سطح پایین (Frameworks, Databases, UI) هستند.
- قانون طلایی این معماری: جهت وابستگی‌های کدهای منبع همیشه باید رو به داخل و به سمت سیاست‌های سطح بالا باشد.
- تناقض معماری و حل آن به کمک وارونگی در عمل یک تناقض پیش می‌آید: یک Use Case سطح بالا (مثلاً RegisterUser) برای انجام کار خود نیاز دارد داده‌ای را ذخیره کند یا ایمیلی بفرستد، که این‌ها عملیاتی سطح پایین و مربوط به ابزارهای بیرونی هستند.
- Clean Architecture این تناقض را دقیقاً با الهام از DIP حل می‌کند: به جای اینکه Use Case مستقیماً به کدهای دیتابیس وابسته شود، یک اینترفیس در داخل لایه خود تعریف می‌کند. لایه دیتابیس در بیرون، این اینترفیس را پیاده‌سازی می‌کند. در نتیجه، وابستگی زمان کامپایل کاملاً وارونه شده و به سمت داخل متمایل می‌شود، در حالی که جریان داده در زمان اجرا به سمت بیرون است.

مشکلات ساختاری که با رعایت این اصل حل می‌شوند

- **Coupling شدید:** بدون DIP، هرگونه تغییر در جزئیات دیتابیس (مثلاً مهاجرت از PostgreSQL به MongoDB) یا تغییر در ابزارهای بیرونی، منطق اصلی کسب‌وکار را خراب می‌کند. DIP هسته سیستم را از نوسانات زیرساخت ایزوله می‌کند.
- **تست‌پذیری ضعیف:** وقتی منطق برنامه به سیستم‌های واقعی متصل باشد، نوشتن Unit Test سریع غیرممکن است. با وابستگی به انتزاع‌ها، می‌توان در محیط تست به راحتی ساختارهای فرضی را تزریق کرد.
- **اختلال در توسعه موازی:** تیم توسعه لایه تجاری می‌تواند بدون منتظر ماندن برای نهایی شدن ساختار دیتابیس یا سرویس‌های شبکه، کدهای خود را به طور مستقل بنویسد و تست کند.

نحوه متصل شدن دو لایه به یکدیگر در زمان اجرا

بر اساس قانون وابستگی، لایه‌های درونی هیچ دیدی به لایه‌های بیرونی ندارند. اما در زمان اجرا، جریان کنترل برای دسترسی به دیتابیس باید از لایه تجاری خارج شود. برای حل این تناقض بدون مخدوش شدن مرزهای معماری، از الگوی Dependency Injection استفاده می‌شود. در این روش، لایه بالا نیازمندی خود را به صورت یک اینترفیس اعلام می‌کند. سپس در بیرونی‌ترین نقطه برنامه یعنی Composition Root (مانند تابع main)، پیاده‌سازی واقعی دیتابیس ساخته شده و به لایه بالا تزریق می‌شود. مراحل کدی این اتصال به شرح زیر است:

تعریف انتزاع و مدل در لایه سطح بالا (Core)

```

1 package usecases
2
3 // Domain Entity -
4 type User struct {
5     ID    string
6     Name  string
7 }
8
9 // UserRepository -
10 type UserRepository interface {
11     GetByID(id string) (*User, error)
12 }

```

پیاده‌سازی مستقل در لایه سطح پایین (Infrastructure)

```

1 package infrastructure
2
3 import (
4     "database/sql"
5     "errors"
6     "usecases"
7 )
8
9 type DBUser struct {
10     ID          string
11     FullName    string
12     CreatedAt   string
13 }
14
15 type SQLUserRepository struct {
16     db *sql.DB
17 }
18
19 func NewSQLUserRepository(db *sql.DB) *SQLUserRepository {
20     return &SQLUserRepository{db: db}
21 }
22
23 func (r *SQLUserRepository) GetByID(id string) (*usecases.User, error) {
24     query := "SELECT * FROM users WHERE id = $1;"
25
26     var dbUser DBUser
27
28     err := r.db.QueryRow(query, id).Scan(&dbUser.ID, &dbUser.FullName, &dbUser.CreatedAt)
29     if err != nil {
30         if errors.Is(err, sql.ErrNoRows) {
31             return nil, nil
32         }
33         return nil, err
34     }
35
36     domainUser := &usecases.User{
37         ID:    dbUser.ID,
38         Name:  dbUser.FullName,
39     }
40
41     return domainUser, nil
42 }

```

تزریق وابستگی در نقطه ورود (Composition Root)

```
1 package main
2
3 import (
4     "database/sql"
5     _ "github.com/lib/pq"
6     "infrastructure"
7     "usecases"
8 )
9
10 func main() {
11     dbConn, err := sql.Open("postgres", "postgres://user:pass@localhost/db")
12     if err != nil {
13         panic(err)
14     }
15     defer dbConn.Close()
16
17     userRepoImpl := infrastructure.NewSQLUserRepository(dbConn)
18
19     userUseCase := usecases.NewUserUseCase(userRepoImpl)
20
21     user, err := userUseCase.Execute("123")
22 }
```

پاسخ مسئله‌ی ۳.

چارچوب‌محور بودن به این معناست که معماری، ساختار پوشه‌ها، و منطق تجاری نرم‌افزار کاملاً بر اساس قوانین، ساختار و امکانات یک فریمورک خاص (مانند Django Spring Laravel, NestJS) شکل بگیرد. در این حالت، ابزارها و فریمورک‌ها در قلب سیستم قرار دارند و کدهای هسته برنامه به شدت به کامپوننت‌های داخلی فریمورک وابسته می‌شوند.

وابستگی شدید به یک فریمورک مشکلات جدی در طول عمر پروژه ایجاد می‌کند:

- **وابستگی شدید:** منطق بیزینس مستقیماً کلاس‌های فریمورک را ارث‌بری می‌کند یا از ابزارهای آن استفاده می‌کند. اگر فریمورک تغییر کند یا منسوخ شود، بخش زیادی از سیستم باید بازنویسی شود.
 - **دشواری در تست‌نویسی:** برای نوشتن یک Unit Test ساده روی منطق بیزینس، مجبورید ساختار فریمورک، وب‌سرور یا دیتابیس متصل به آن را شبیه‌سازی یا اجرا کنید که سرعت و دقت تست‌ها را کاهش می‌دهد.
 - **کابوس به‌روزرسانی:** با معرفی نسخه‌های جدید فریمورک، به‌روزرسانی پروژه بسیار زمان‌بر، پرهزینه و خطرناک خواهد بود.
 - **محدودیت در انعطاف‌پذیری:** توسعه‌دهنده مجبور است کدهای خود را با ساختار پیشنهادی فریمورک هماهنگ کند، حتی اگر آن ساختار برای حل مسئله‌ی خاص بیزینس بهینه نباشد.
- Clean Architecture با تغییر جهت وابستگی‌ها و قرار دادن منطق بیزینس در مرکز، این مشکلات را حل می‌کند:

- **قانون وابستگی:** در این معماری، لایه‌ها به صورت دایره‌های متمرکز طراحی می‌شوند. قانون طلایی این است: وابستگی‌ها فقط باید به سمت داخل باشند. یعنی لایه‌های درونی (بیزینس) هیچ اطلاعی از لایه‌های بیرونی (فریمورک، دیتابیس) ندارند. فریمورک صرفاً یک ابزار در بیرونی‌ترین لایه است.
- **جداسازی منطق تجاری:**
 - Entities: قوانین کلی و مستقل بیزینس را کپسوله‌سازی می‌کنند.
 - Use Cases: منطق اختصاصی هر عملیات سیستم را پیاده‌سازی می‌کنند.
- این دو لایه مرکزی کاملاً با کد خالص بدون وارد کردن پکیج‌ها یا دکوراتورهای فریمورک نوشته می‌شوند.
- **اصل وارونگی وابستگی:** اگر لایه داخلی نیاز به ارتباط با یک ابزار بیرونی (مانند دیتابیس یا سرویس پیام‌رسان) داشته باشد، مستقیماً به آن وصل نمی‌شود؛ بلکه یک اینترفیس در لایه داخلی تعریف می‌کند و فریمورک در لایه بیرونی موظف است آن را پیاده‌سازی کند.

با استفاده از Clean Architecture فریمورک تبدیل به یک جزئیات می‌شود، نه محور سیستم. شما می‌توانید لایه وب یا دیتابیس فریمورک را بدون دست زدن به کدهای اصلی بیزینس تعویض یا به‌روزرسانی کنید.

پاسخ مسئله‌ی ۴.

پاسخ مسئله‌ی ۵.

شباهت‌های بین دو الگوی Clean Architecture و MVVM

با توجه به مباحث مطرح‌شده در متن مقاله و Case Study، شباهت‌های بنیادی و ساختاری زیر بین این دو الگو دیده می‌شود:

- **Separation of Concerns**: هر دو الگو با هدف تفکیک وظایف لایه‌های مختلف سیستم طراحی شده‌اند. الگوی MVVM رابط کاربری را از منطق نمایش جدا می‌کند و Clean Architecture کل نرم‌افزار را به لایه‌های مستقل تقسیم می‌نماید.
- **Testability**: در هر دو ساختار، بستری عالی برای Unit Testing مهیا شده است. در MVVM، لایه ViewModel مستقل از UI تست می‌شود و در Clean Architecture نیز هر لایه به طور جداگانه قابل ارزیابی است.
- **Modularity**: هر دو الگو کدها را به بخش‌های مجزا و مستقل تقسیم می‌کنند که توسعه موازی و مدیریت پروژه را تسهیل می‌کند.
- **پیچیدگی و سربار در پروژه‌های کوچک**: مقاله به صراحت اشاره می‌کند که هر دو الگو برای پروژه‌های ساده و کوچک، به دلیل ایجاد کدهای تکراری زیاد و پیچیدگی اولیه، نوعی زیاده‌روی به شمار می‌روند.

نحوه ادغام و ترکیب دو الگو توسط نویسنده

نویسنده مقاله برای بهره‌گیری از نقاط قوت هر دو ساختار، آن‌ها را به شکل یک معماری هیبریدی منسجم ادغام کرده است. نقطه عطف این ترکیب در لایه واسط قرار دارد. نویسنده ViewModel (از الگوی MVVM) را دقیقاً هم‌تراز و معادل لایه Interface Adapter (از الگوی Clean Architecture) قرار داده است با این کار، وظیفه مدیریت وضعیت نمایش را بر عهده می‌گیرد در حالی که با لایه‌های درونی‌تر برای اجرای منطق تجاری در ارتباط است.

جریان ارتباطی لایه‌ها در لایه‌بندی ترکیبی:

۱. **View (لایه UI)**: بیرونی‌ترین سطح که با کاربر در تعامل است و از طریق فرآیند Data Binding به ویژگی‌های ViewModel متصل می‌شود.
۲. **ViewModel (لایه Presentation)**: نقش واسط را ایفا کرده، ورودی‌های View را گرفته و Use Case متناسب را فراخوانی می‌کند.
۳. **Use Cases (لایه Business Logic)**: حاوی منطق‌های خاص هر قابلیت برنامه (مانند افزودن یا حذف یک Task) است.
۴. **Entities (لایه Core Rules)**: شامل اشیاء پایه و قوانین بنیادی کسب‌وکار است که به هیچ فریم‌ورک خارجی وابستگی ندارد.
۵. **Data Layer**: وظیفه تعامل با پایگاه داده محلی یا API‌های سرور و مدیریت ماندگاری داده‌ها را بر عهده دارد.

پروژه‌های مناسب برای استفاده از این الگوی ترکیبی و دلایل آن

بر اساس تحلیل مزایا و معایب در متن مقاله، این معماری ترکیبی برای پروژه‌هایی با ویژگی‌های زیر بیشترین بازدهی را دارد:

- **پروژه‌های بزرگ و سازمانی:** به دلیل ساختار کاملاً مدولار، این الگو اجازه می‌دهد تیم‌های بزرگ بدون تداخل با یکدیگر روی بخش‌های مختلف پروژه کار کنند.
 - **پروژه‌های با طول عمر بالا و نیازمند نگهداری:** از آنجا که لایه‌ها به وسیله اینترفیس‌ها از هم ایزوله شده‌اند، تغییر در یک لایه (مثلاً تعویض کتابخانه شبکه یا تغییر دیتابیس) تأثیری روی منطق اصلی برنامه نخواهد گذاشت.
 - **پروژه‌های مبتنی بر TDD:** برای برنامه‌هایی که پوشش تست بالایی نیاز دارند، تفکیک دقیق لایه‌ها امکان نوشتن تست‌های واحد پایدار و مستقل را فراهم می‌سازد.
- پیاده‌سازی این ساختار نیازمند تعریف اینترفیس‌های متعدد، کلاس‌های مختلف و تنظیمات اولیه پیچیده‌ای است در پروژه‌های کوچک، زمان و هزینه‌ای که صرف نوشتن این کدهای اضافه می‌شود، توجیه اقتصادی و فنی ندارد.